

combined paper plots

April 14, 2026

1 Pre Process

```
[76]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

models_scores_path = "C:
↳\\Users\\matan\\master_thesis\\rl_envs\\procgen\\results\\simple_eval100.
↳csv" #"../..../results/models_results.csv"
models_scores = pd.read_csv(models_scores_path)
models_mean_scores = models_scores.groupby('model_index')['total_reward'].mean()
models_mean_dict = {str(k): v for k, v in models_mean_scores.to_dict().items()}

user_question_mapping = {
    'ES_understandable': "The explanations of the agent's paths were clear and
↳easy to understand",
    'ES_overwhelming': "The explanations felt too detailed and overwhelming",
    'ES_feedback_related': "The explanations helped me see how my feedback
↳changed the agent's behavior",
    'ES_helpful': "The visual demonstrations of policy changes helped me
↳understand the differences between agents",
    'ES_improvement': "My feedback helped improve the agent's performance",
    'ES_combined': "Combined score of the explanations' satisfaction",
    'ai': "How familiar are you with Artificial Intelligence?",
    'AI_1': "I believe AI will improve my life",
    'AI_2': "I believe that AI will improve my work",
    'AI_3': "I think I will use AI technology in the future",
    'AI_4': "I think AI technology is a threat to humans",
    'AI_5': "I think AI technology is positive for humanity",
}

minigrid_models_mean_scores = {6: ('NoRedLH1_0612', 10.87),
                                5: ('AllColorsLH_0617', 9.39),
                                4: ('OnlyBlueLH_0706', 8.84),
                                3: ('OnlyBlueLL_0625', 7.49),
                                2: ('AllColorsLL1_0526', 6.48),
```

```
1: ('OnlyGreenLL_0429', 1.88)}
```

```
minigrid_user_results = pd.read_csv('user_results_for_combined_minigrid_df.csv')
fruitbot_user_results = pd.read_csv('user_results_for_combined_fruitbot_df.csv')
```

```
# choices_results = pd.read_csv('choices_final_results.csv')
```

```
[77]: choices_results = pd.read_csv('choices_results_combined.csv')
      user_results = pd.read_csv('user_results_combined.csv')
```

```
[78]: choices_results['group_label'].value_counts()
```

```
[78]: group_label
      Random          446
      Control        423
      Salient-Contrast 418
      Same           406
      Name: count, dtype: int64
```

```
[79]: user_results['group_label'].value_counts()
```

```
[79]: group_label
      Salient-Contrast 129
      Random          126
      Control          122
      Same            117
      Name: count, dtype: int64
```

2 Plotting

```
[80]: import pandas as pd
      import matplotlib.pyplot as plt
      import seaborn as sns
      import numpy as np
      from scipy.stats import mannwhitneyu, f_oneway, kruskal
      import itertools, warnings
      warnings.filterwarnings("ignore")
      %matplotlib inline
```

```
[81]: # helper functions: plot_barplot, mannwhitneyu_test, anova_test

      colors_palette = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6']
      group_names = ['Control', 'Random', 'Same', 'Salient-Contrast']
      out_dir = 'paper_figures_pdf'
      fruitbot_agent_rank_map = {1: 2, 2: 1, 3: 3, 4: 4, 5: 5, 6: 6} # Map prev/
      ↪ updated agent to rank (example mapping)
```

```

def plot_barplot(df, x_col, y_col, hue=None, x_val_names=None,
                 figsize=(6, 5), estimator=lambda x: np.mean(x) * 100,
errorbar=('ci', 95),
                 palette=None, x_label='Group', y_label='%', order=None,
                 bar_label_fmt='%.1f%%', ylim_bottom=0, title=None,
save_path=None, hue_order=None):
    if order is None:
        order = sorted(list(df[x_col].unique()))
    plt.figure(figsize=figsize)
    ax = sns.barplot(data=df, x=x_col, y=y_col, hue=hue, hue_order=hue_order,
errorbar=estimator,
                    errorbar=errorbar, palette=palette, order=order)
    # for c in ax.containers:
    #     ax.bar_label(c, fmt=bar_label_fmt, label_type='edge', fontsize=13)
    x_val_names = x_val_names or [str(l) for l in order]
    ax.set_xticklabels(x_val_names, rotation=(40 if len(x_val_names[0]) > 15
else 0),
                    ha=('right' if len(x_val_names[0]) > 15 else 'center'),
fontsize=13)
    if x_label: plt.xlabel(x_label, fontsize=14)
    if y_label: plt.ylabel(y_label, fontsize=14)
    plt.yticks(fontsize=13)
    if hue:
        handles, labels = ax.get_legend_handles_labels()

        if hue == 'group_label':
            desired_order = hue_order if hue_order is not None else [g for g in
group_names if g in labels]
            handle_map = dict(zip(labels, handles))
            labels = [l for l in desired_order if l in handle_map]
            handles = [handle_map[l] for l in labels]

        display_labels = labels

        ax.legend(handles=handles, labels=display_labels, fontsize=11,
loc='lower right')
    plt.ylim(bottom=ylim_bottom)
    if title:
        plt.title(title, fontsize=14)
    plt.tight_layout()
    if save_path:
        plt.savefig(f"{out_dir}/{save_path}", bbox_inches='tight')

    # Print the exact values plotted in each bar.

```

```

def _apply_estimator(series):
    if callable(estimator):
        return estimator(series)
    if isinstance(estimator, str):
        est = estimator.lower()
        if est == 'mean':
            return series.mean()
        if est == 'median':
            return series.median()
        if est == 'sum':
            return series.sum()
        if est == 'count':
            return series.count()
        if est == 'min':
            return series.min()
        if est == 'max':
            return series.max()
    return series.mean()

if hue:
    plot_values = (
        df.groupby([x_col, hue])[y_col]
        .apply(_apply_estimator)
        .reset_index(name='plot_value')
        .pivot(index=x_col, columns=hue, values='plot_value')
    )
    plot_values = plot_values.reindex(index=order)
    if hue_order is not None:
        plot_values = plot_values.reindex(columns=hue_order)
else:
    plot_values = df.groupby(x_col)[y_col].apply(_apply_estimator).
    ↪ rename('plot_value')
    plot_values = plot_values.reindex(order)

print('\nPlot values:')
print(plot_values.round(3).to_string())

plt.show()

def mannwhitneyu_test(df, y_col, x_col, test_type='two-sided'):
    levels = sorted(df[x_col].unique())
    rows = []
    for a, b in itertools.combinations(levels, 2):
        g1, g2 = df[df[x_col]==a][y_col].dropna(), df[df[x_col]==b][y_col].
        ↪ dropna()
        u, p = mannwhitneyu(g1, g2, alternative=test_type)
        rows.append({'group_1': a, 'group_2': b, 'U': u, 'p': round(p, 4)})

```

```

print(pd.DataFrame(rows).to_string(index=False))

def anova_test(df, y_col, x_col):
    groups = [g[y_col].dropna().values for _, g in df.groupby(x_col)]
    f, pf = f_oneway(*groups)
    h, pk = kruskal(*groups)
    print(f"ANOVA F={f:.4f}, p={pf:.4f} | Kruskal-Wallis H={h:.4f}, p={pk:.4f}")

```

2.1 Correct Choice

```

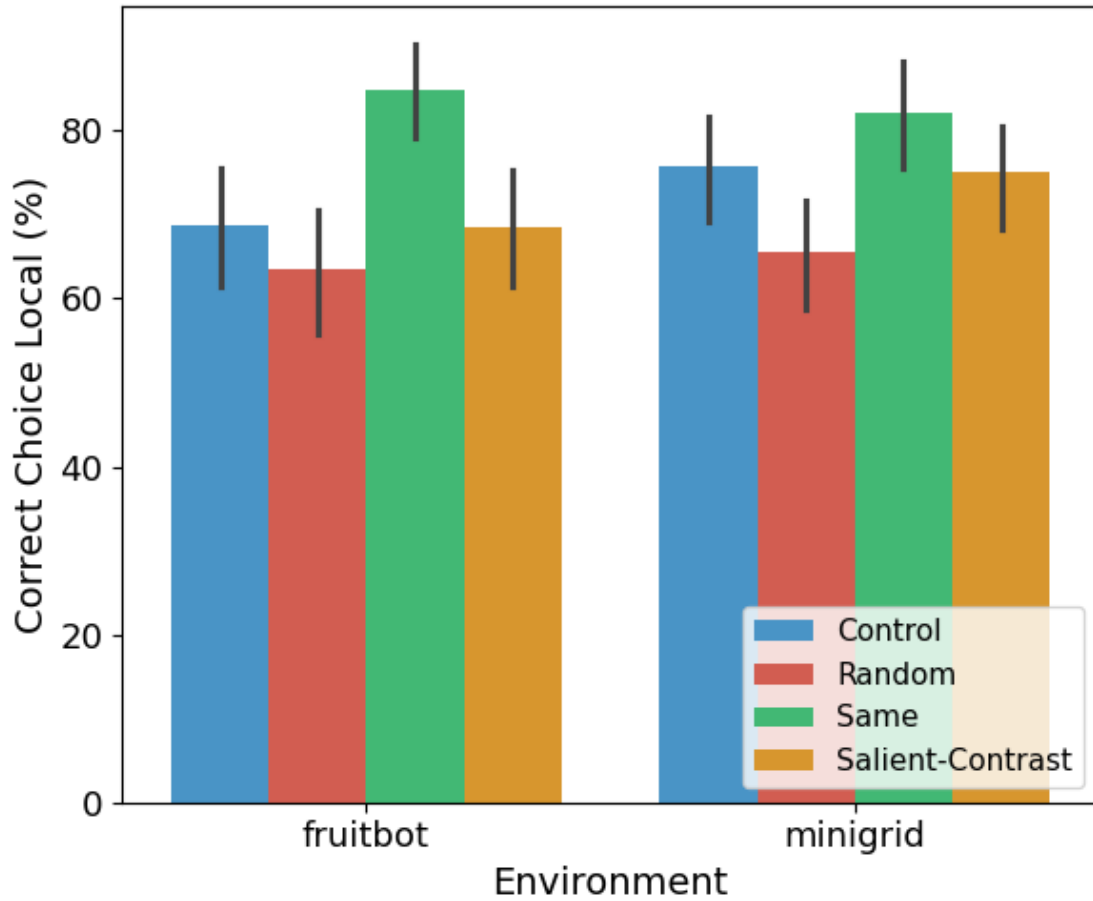
[82]: # Plot: Correct Choice (Local) by Group
plot_barplot(user_results, 'environment', 'mean_good_choice_local',
             hue='group_label', hue_order=group_names, x_label='Environment',
             y_label='Correct Choice Local (%)', palette=colors_palette,
             bar_label_fmt='%.0f', save_path='correct_choice_local_by_group.pdf')

for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env}:")
    anova_test(user_results[user_results['environment'] == env],
               'mean_good_choice_local', 'group_label')
    mannwhitneyu_test(user_results[user_results['environment'] == env],
                     'mean_good_choice_local', 'group_label', test_type='two-sided')

```

Plot values:

group_label	Control	Random	Same	Salient-Contrast
environment				
fruitbot	68.679	63.400	84.879	68.571
minigrid	75.700	65.548	82.097	75.000



statistic tests for minigrid:

ANOVA $F=4.4751$, $p=0.0044$ | Kruskal-Wallis $H=15.2891$, $p=0.0016$

group_1	group_2	U	p
Control	Random	3178.5	0.0242
Control	Salient-Contrast	2555.0	0.8784
Control	Same	1773.5	0.0729
Random	Salient-Contrast	2242.0	0.0381
Random	Same	1510.0	0.0002
Salient-Contrast	Same	1851.5	0.0522

statistic tests for fruitbot:

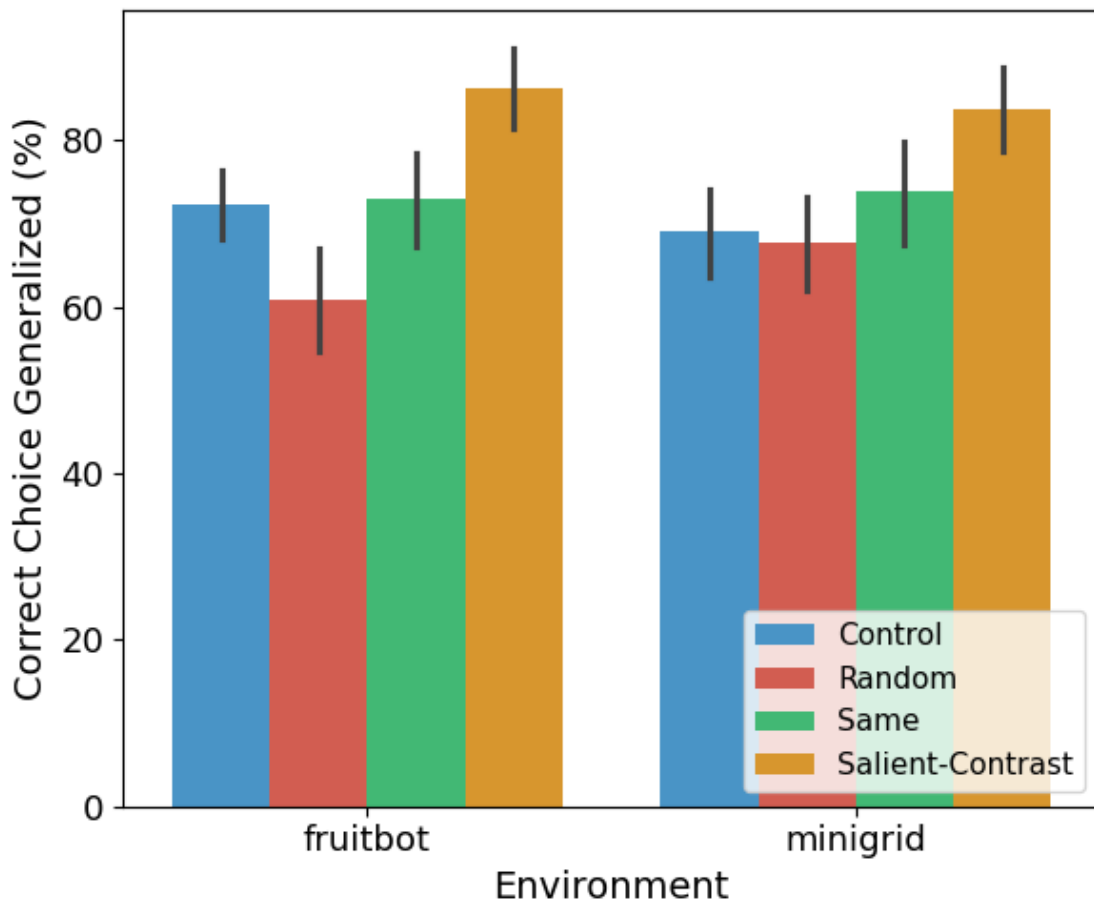
ANOVA $F=7.1260$, $p=0.0001$ | Kruskal-Wallis $H=21.1580$, $p=0.0001$

group_1	group_2	U	p
Control	Random	1482.5	0.2944
Control	Salient-Contrast	1520.0	0.8266
Control	Same	918.5	0.0005
Random	Salient-Contrast	1271.0	0.4094
Random	Same	738.0	0.0000
Salient-Contrast	Same	993.5	0.0007

```
[83]: # Plot: Correct Choice (Generalized) by Group
plot_barplot(user_results, 'environment', 'mean_good_choice_global',
             hue='group_label', errorbar=('ci', 95), x_label='Environment',
             hue_order=group_names, y_label='Correct Choice Generalized (%)',
             palette=colors_palette, bar_label_fmt='%.0f',
             save_path='correct_choice_generalized_by_group.pdf')
for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env}:")
    mannwhitneyu_test(user_results[user_results['environment'] == env],
                      'mean_good_choice_global', 'group_label')
```

Plot values:

group_label	Control	Random	Same	Salient-Contrast
environment				
fruitbot	72.327	60.933	72.879	86.220
minigrid	69.034	67.719	73.817	83.744



statistic tests for minigrid:

group_1	group_2	U	p
Control	Random	2620.5	0.9968
Control	Salient-Contrast	1533.5	0.0000
Control	Same	1795.5	0.1067
Random	Salient-Contrast	1740.5	0.0000
Random	Same	2017.5	0.1396
Salient-Contrast	Same	2823.5	0.0090

statistic tests for fruitbot:

group_1	group_2	U	p
Control	Random	1758.5	0.0037
Control	Salient-Contrast	818.0	0.0000
Control	Same	1490.0	0.8415
Random	Salient-Contrast	567.5	0.0000
Random	Same	999.5	0.0146
Salient-Contrast	Same	2095.0	0.0006

2.2 Correct Choice seperated by improve dir

```
[84]: # Plot: Correct Choice (Local) by Update Direction - Same vs Contrast
envs = ['minigrid', 'fruitbot']
fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

for ax, env in zip(axes, envs):
    env_df = choices_results[
        (choices_results['group_label'].isin(['Same', 'Salient-Contrast'])) &
        (choices_results['environment'] == env)
    ].copy()

    sns.barplot(
        data=env_df,
        x='improved_from_prev_feedback',
        y='good_choice_local',
        hue='group_label',
        estimator=lambda x: np.mean(x) * 100,
        errorbar=('ci', 95),
        order=[0, 1],
        hue_order=['Same', 'Salient-Contrast'],
        palette=['#3498db', '#e74c3c'],
        ax=ax
    )

    # for c in ax.containers:
    #     ax.bar_label(c, fmt='%.0f%%', label_type='edge', fontsize=10)

    ax.set_title(env.capitalize(), fontsize=13)
    ax.set_xlabel('Policy Update')
    ax.set_ylabel('Correct Choice Local (%)' if ax is axes[0] else '')
```

```

ax.set_xticklabels(['Negative', 'Positive'])

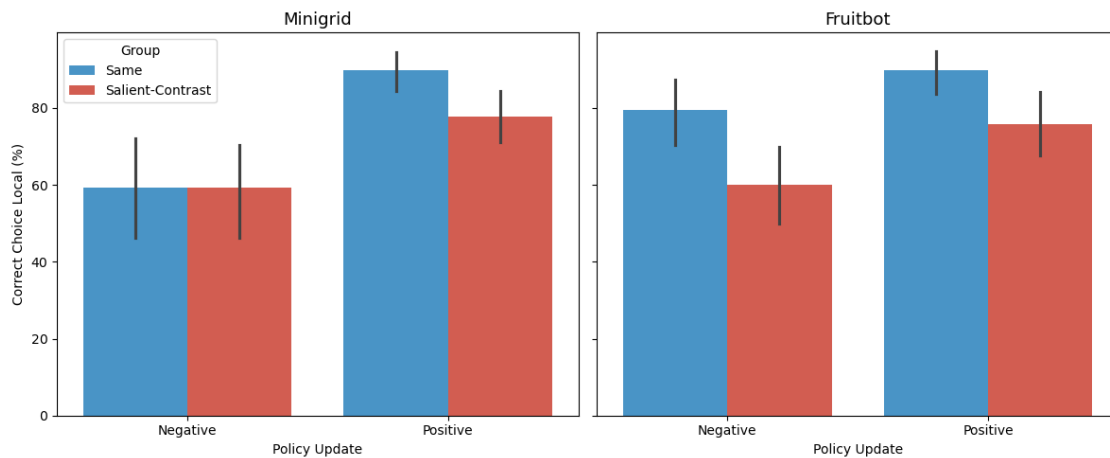
if ax is not axes[0]:
    ax.legend_.remove()

axes[0].legend(title='Group', fontsize=10)
plt.tight_layout()

plt.savefig(f"{out_dir}/correct_choice_local_by_update_direction.pdf",
    ↳bbox_inches='tight')
plt.show()

# Keep plot_df for statistical tests below (seperate to each environment)
for env in ['minigrid', 'fruitbot']:
    plot_df = choices_results[
        (choices_results['group_label'].isin(['Same', 'Salient-Contrast'])) &
        (choices_results['environment'] == env)
    ].copy()
    print(f"--- Statistical tests for {env} ---")
    print("--- Positive updates ---")
    mannwhitneyu_test(plot_df[plot_df['improved_from_prev_feedback']==1],
    ↳'good_choice_local', 'group_label')
    print("--- Negative updates ---")
    mannwhitneyu_test(plot_df[plot_df['improved_from_prev_feedback']==0],
    ↳'good_choice_local', 'group_label')

```



```

--- Statistical tests for minigrid ---
--- Positive updates ---
      group_1 group_2      U      p
Salient-Contrast  Same 10726.5 0.0042
--- Negative updates ---

```

```

        group_1 group_2      U      p
Salient-Contrast    Same 1458.0 1.0
--- Statistical tests for fruitbot ---
--- Positive updates ---
        group_1 group_2      U      p
Salient-Contrast    Same 5445.0 0.0058
--- Negative updates ---
        group_1 group_2      U      p
Salient-Contrast    Same 3186.0 0.0047

```

```
[85]: choices_results.groupby(['group_label',
    ↪ 'improved_from_prev_feedback'])['good_choice_local'].mean()
```

```
[85]: group_label      improved_from_prev_feedback
Control              0                      0.132353
              1                      1.000000
Random              0                      0.517483
              1                      0.686469
Salient-Contrast    0                      0.597222
              1                      0.770073
Same                0                      0.718310
              1                      0.897727
Name: good_choice_local, dtype: float64
```

```
[86]: # # Plot: Correct Choice (Generalized) by Update Direction - Same vs Contrast
# plot_df = choices_results[choices_results['group_label'].isin(['Same',
    ↪ 'Salient-Contrast'])].copy()
# for env in ['minigrid', 'fruitbot']:
#     env_df = plot_df[plot_df['environment'] == env]
#     plot_barplot(env_df, 'improved_from_prev_mean', 'good_choice_global',
    ↪ hue='group_label',
#         y_label=f'Correct Choice (Generalized) % - {env.
    ↪ capitalize()}', x_label='Policy Update',
#         x_val_names=['Negative', 'Positive'], palette=['#f39c12',
    ↪ '#e74c3c', '#3498db', '#2ecc71'])
# plot_barplot(plot_df, 'improved_from_prev_mean', 'good_choice_global',
#     y_label='Correct Choice (Generalized) %', x_label='Policy
    ↪ Update',
#     x_val_names=['Negative', 'Positive'], hue='group_label',
#     palette=['#f39c12', '#e74c3c', '#3498db', '#2ecc71'])
# print("--- Positive updates ---")
# mannwhitneyu_test(plot_df[plot_df['improved_from_prev_mean']==1],
    ↪ 'good_choice_global', 'group_label')
# print("--- Negative updates ---")
# mannwhitneyu_test(plot_df[plot_df['improved_from_prev_mean']==0],
    ↪ 'good_choice_global', 'group_label')
```

```

# Plot: Correct Choice (Local) by Update Direction - Same vs Contrast
envs = ['minigrid', 'fruitbot']
fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)

for ax, env in zip(axes, envs):
    env_df = choices_results[
        (choices_results['group_label'].isin(['Same', 'Salient-Contrast'])) &
        (choices_results['environment'] == env)
    ].copy()

    sns.barplot(
        data=env_df,
        x='improved_from_prev_mean',
        y='good_choice_global',
        hue='group_label',
        estimator=lambda x: np.mean(x) * 100,
        errorbar=('ci', 95),
        order=[0, 1],
        hue_order=['Same', 'Salient-Contrast'],
        palette=['#3498db', '#e74c3c'],
        ax=ax
    )

    # for c in ax.containers:
    #     ax.bar_label(c, fmt='%.0f%%', label_type='edge', fontsize=10)

    ax.set_title(env.capitalize(), fontsize=13)
    ax.set_xlabel('Policy Update')
    ax.set_ylabel('Correct Choice Generalized (%)' if ax is axes[0] else '')
    ax.set_xticklabels(['Negative', 'Positive'])

    if ax is not axes[0]:
        ax.legend_.remove()

axes[0].legend(title='Group', fontsize=10)
plt.tight_layout()
plt.savefig(f"{out_dir}/correct_choice_generalized_by_update_direction.pdf",
           bbox_inches='tight')
plt.show()

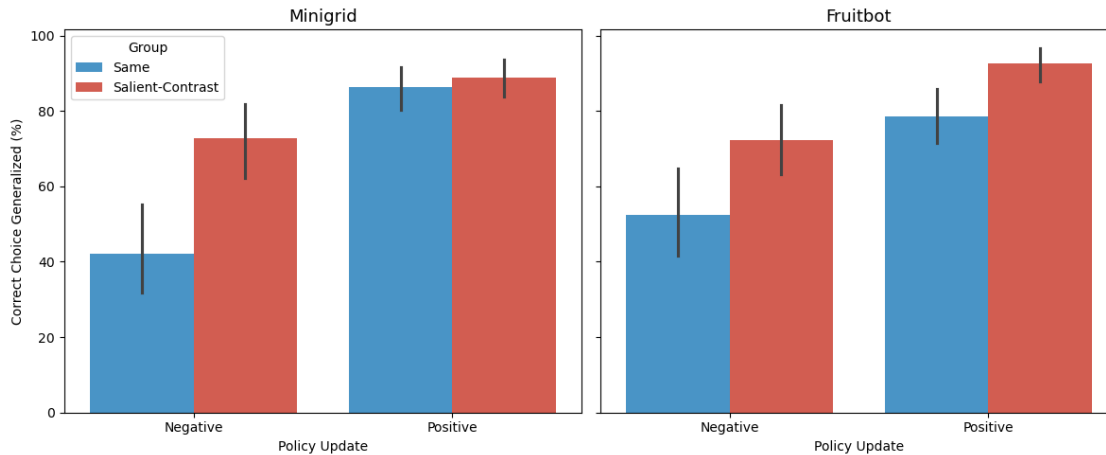
for env in ['minigrid', 'fruitbot']:
    plot_df = choices_results[(choices_results['group_label'].isin(['Same',
        'Salient-Contrast'])) & (choices_results['environment'] == env) ].copy()
    print(f"--- Statistical tests for {env} ---")
    print("--- Positive updates ---")

```

```

mannwhitneyu_test(plot_df[plot_df['improved_from_prev_mean']==1],
↪ 'good_choice_global', 'group_label')
print("--- Negative updates ---")
mannwhitneyu_test(plot_df[plot_df['improved_from_prev_mean']==0],
↪ 'good_choice_global', 'group_label')

```



```

--- Statistical tests for minigrid ---
--- Positive updates ---
      group_1 group_2      U      p
Salient-Contrast  Same 9669.0 0.5395
--- Negative updates ---
      group_1 group_2      U      p
Salient-Contrast  Same 3472.0 0.0002
--- Statistical tests for fruitbot ---
--- Positive updates ---
      group_1 group_2      U      p
Salient-Contrast  Same 9740.0 0.0015
--- Negative updates ---
      group_1 group_2      U      p
Salient-Contrast  Same 2965.5 0.0143

```

2.3 Agent Rating

```

[87]: # Plot: user Evaluation of Final Agent Control vs Assessment-Loop

plot_df = user_results.copy()
plot_df['group_binary'] = np.where(plot_df['group_label'].isin(['Control']),
↪ 'Control', 'Assessment-Loop')

plot_barplot(
    plot_df,

```

```

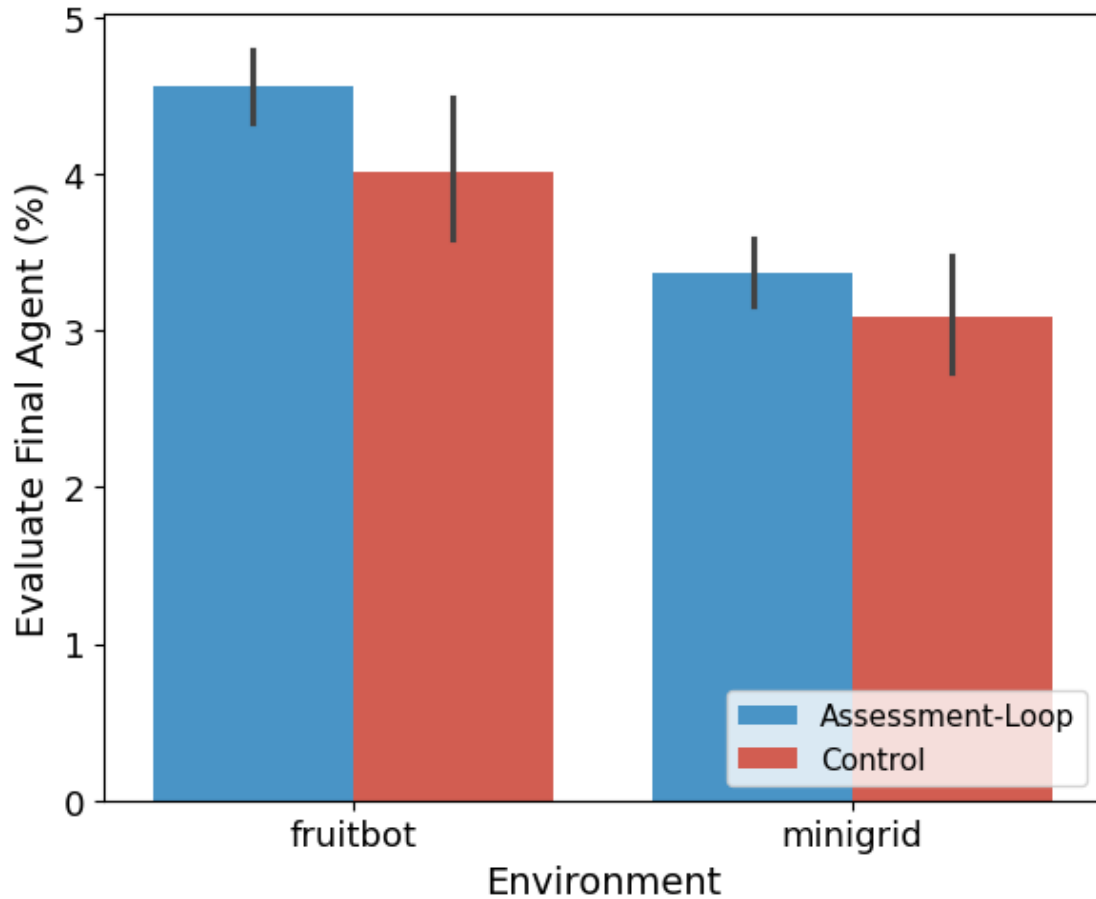
        'environment',
        'evaluate_final_agent',
        hue='group_binary',
        estimator='mean',
        y_label='Evaluate Final Agent (%)',
        x_label='Environment',
        palette=['#3498db', '#e74c3c'],
        bar_label_fmt='%.2f',
        save_path='evaluate_final_agent_control_vs_assessment_loop.pdf'
    )

for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env} on evaluate_final_agent:")
    mannwhitneyu_test(plot_df[plot_df['environment'] == env],
        ↪ 'evaluate_final_agent', 'group_binary', test_type='greater')

```

Plot values:

group_binary	Assessment-Loop	Control
environment		
fruitbot	4.559	4.019
minigrid	3.374	3.087



```

statistic tests for minigrid on evaluate_final_agent:
  group_1 group_2      U      p
Assessment-Loop Control 8161.5 0.0606
statistic tests for fruitbot on evaluate_final_agent:
  group_1 group_2      U      p
Assessment-Loop Control 4997.0 0.0284

```

```
[88]: user_question_mapping
```

```
[88]: {'ES_understandable': "The explanations of the agent's paths were clear and easy
to understand",
      'ES_overwhelming': 'The explanations felt too detailed and overwhelming',
      'ES_feedback_related': "The explanations helped me see how my feedback changed
the agent's behavior",
      'ES_helpful': 'The visual demonstrations of policy changes helped me understand
the differences between agents',
      'ES_improvement': "My feedback helped improve the agent's performance",
      'ES_combined': "Combined score of the explanations' satisfaction",

```

```
'ai:': 'How familiar are you with Artificial Intelligence?',
'AI_1': 'I believe AI will improve my life',
'AI_2': 'I believe that AI will improve my work',
'AI_3': 'I think I will use AI technology in the future',
'AI_4': 'I think AI technology is a threat to humans',
'AI_5': 'I think AI technology is positive for humanity'}
```

```
[89]: user_results['ES_combined'] = (user_results['ES_understandable'] +
    ↪user_results['ES_helpful'] + # + user_results['ES_improvement']
    user_results['ES_feedback_related'] + (7 -
    ↪user_results['ES_overwhelming'])) / 4

user_results.columns
```

```
[89]: Index(['user_id', 'group', 'trust_the_agent', 'number_of_choices',
    'final_agent_mean_score', 'user_performance', 'age', 'gender',
    'education', 'familiar_with_AI', 'evaluate_final_agent', 'AI_1',
    'ES_understandable', 'AI_2', 'autentication_check', 'AI_3',
    'ES_overwhelming', 'AI_4', 'ES_feedback_related', 'AI_5', 'ES_helpful',
    'group_label', 'final_agent', 'mean_good_choice_local',
    'mean_good_choice_global', 'mean_good_choice_demo', 'environment',
    'ES_improvement', 'mean_agent_rating', 'final_score', 'ES_combined'],
    dtype='object')
```

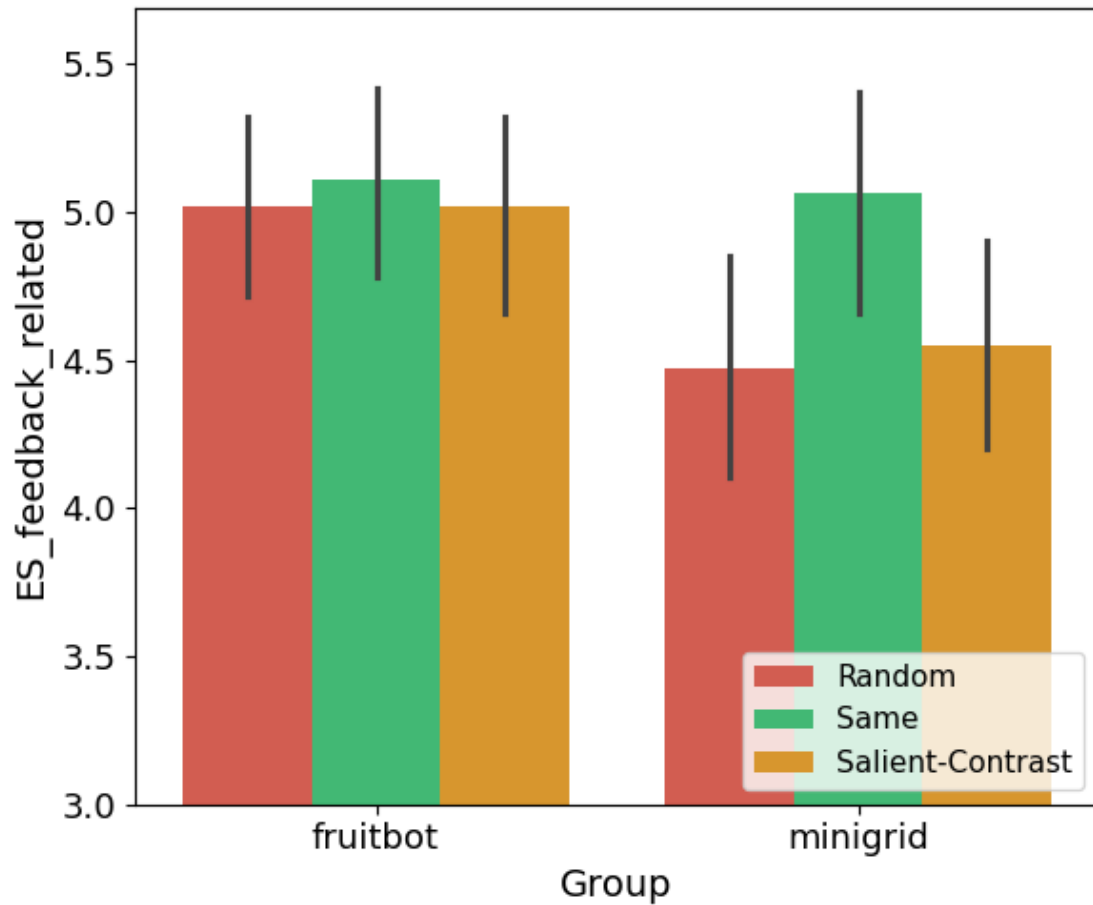
```
[90]: # user satisfaction with the feedback and explanations
satisfaction_list = ['ES_feedback_related', 'ES_understandable',
    ↪'ES_overwhelming', 'ES_helpful', 'ES_improvement', 'ES_combined']
for y_column in satisfaction_list[:]:
    print(user_question_mapping[y_column])
    user_results_p = user_results[user_results[y_column].notnull()].copy()
    user_results_p[y_column] = user_results_p[y_column].astype(int)
    plot_barplot(user_results_p, 'environment', y_column,
        y_label=y_column, palette=colors_palette[1:],
    ↪hue='group_label', hue_order=group_names[1:],
        estimator=lambda x: np.mean(x), bar_label_fmt='%.2f',
    ↪ylim_bottom=3)
    for env in ['minigrid', 'fruitbot']:
        print(f"statistic tests for {env} on {y_column}:")
        mannwhitneyu_test(user_results_p[user_results_p['environment'] == env],
    ↪y_column, 'group_label', test_type='less')
        print("\n" + "-"*50 + "\n")
```

The explanations helped me see how my feedback changed the agent's behavior

Plot values:

```
group_label  Random    Same    Salient-Contrast
environment
```

fruitbot	5.020	5.109	5.018
minigrid	4.474	5.065	4.548



statistic tests for minigrid on ES_feedback_related:

group_1	group_2	U	p
Random	Salient-Contrast	2766.5	0.4892
Random	Same	1890.5	0.0210
Salient-Contrast	Same	1810.5	0.0208

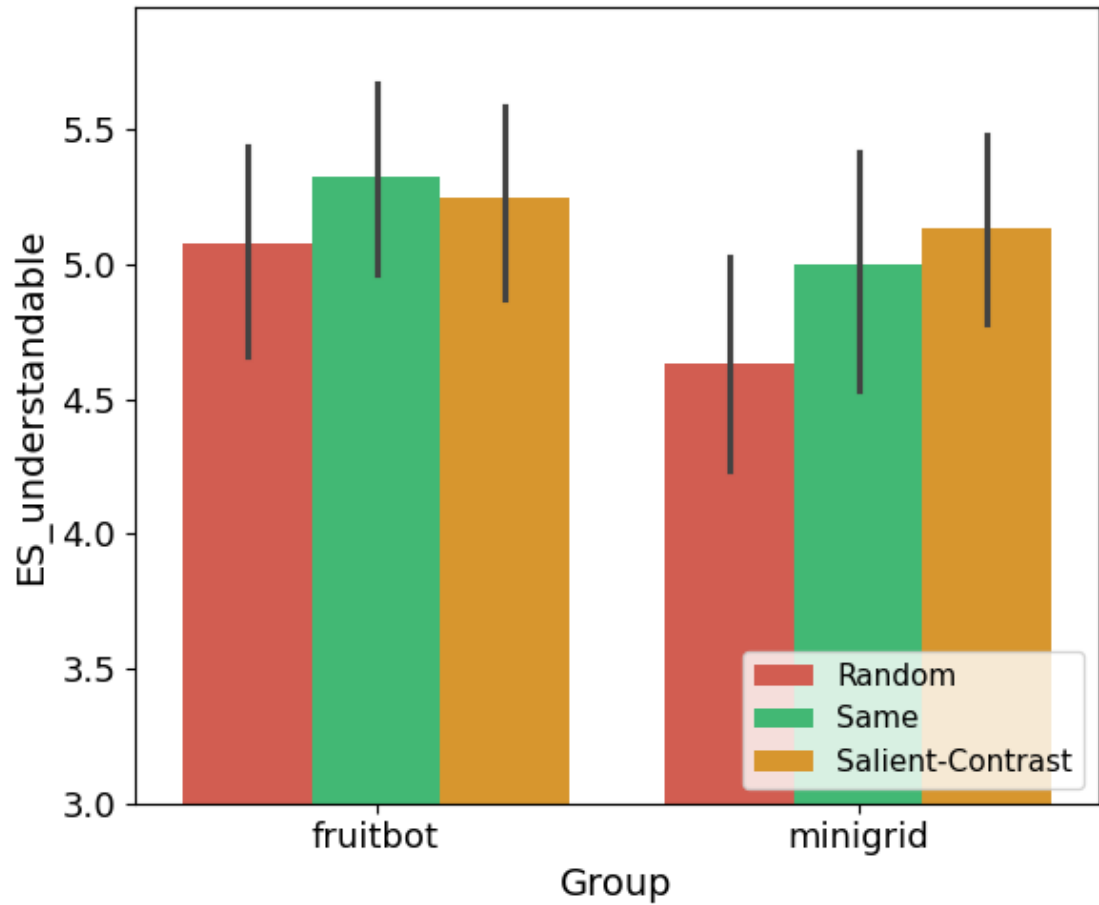
statistic tests for fruitbot on ES_feedback_related:

group_1	group_2	U	p
Random	Salient-Contrast	1336.5	0.3389
Random	Same	1237.0	0.1756
Salient-Contrast	Same	1488.5	0.3769

The explanations of the agent's paths were clear and easy to understand

Plot values:

group_label	Random	Same	Salient-Contrast
environment			
fruitbot	5.080	5.327	5.250
minigrid	4.632	5.000	5.137

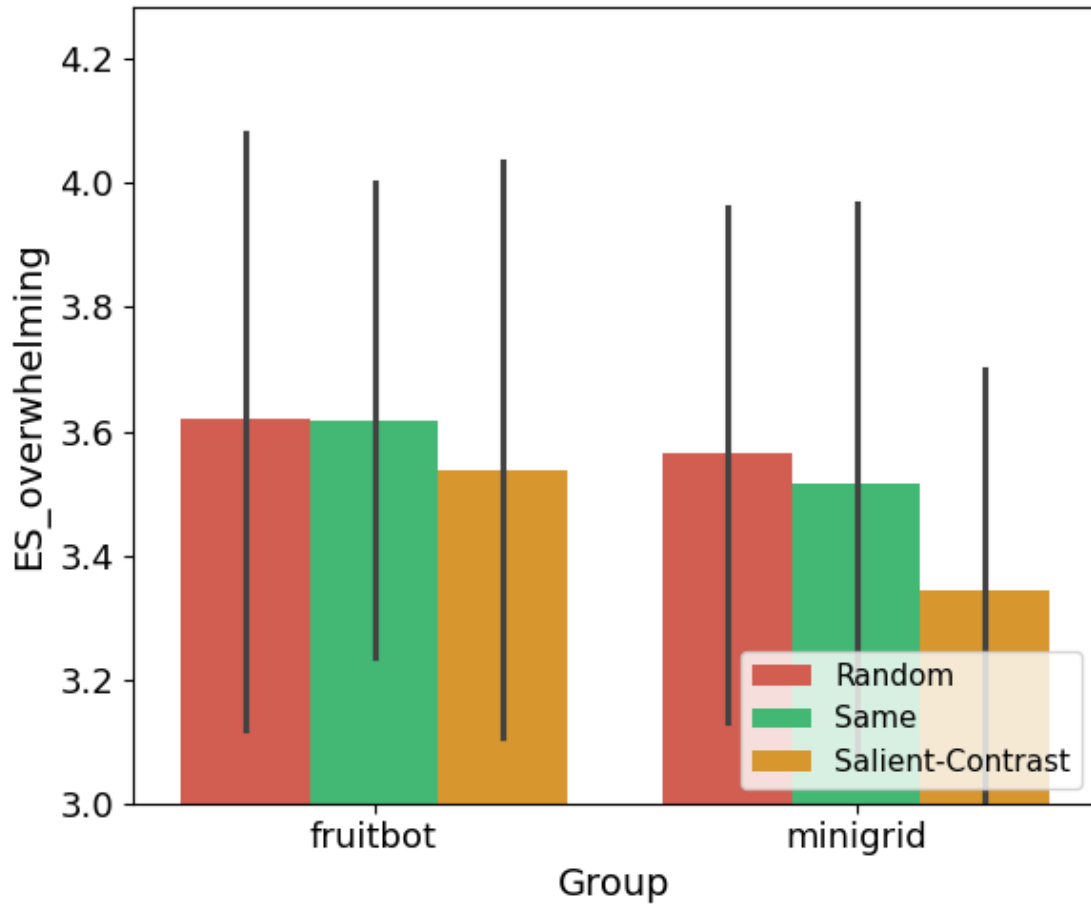


statistic tests for minigrid on ES_understandable:				
group_1	group_2	U	p	
Random	Salient-Contrast	2368.5	0.0576	
Random	Same	2077.5	0.1113	
Salient-Contrast	Same	2313.0	0.5905	
statistic tests for fruitbot on ES_understandable:				
group_1	group_2	U	p	
Random	Salient-Contrast	1346.0	0.3608	
Random	Same	1295.0	0.2959	
Salient-Contrast	Same	1508.0	0.4230	

The explanations felt too detailed and overwhelming

Plot values:

group_label	Random	Same	Salient-Contrast
environment			
fruitbot	3.620	3.618	3.536
minigrid	3.566	3.516	3.342



statistic tests for minigrid on ES_overwhelming:

group_1	group_2	U	p
Random	Salient-Contrast	2936.5	0.7350
Random	Same	2381.5	0.5450
Salient-Contrast	Same	2136.0	0.2853

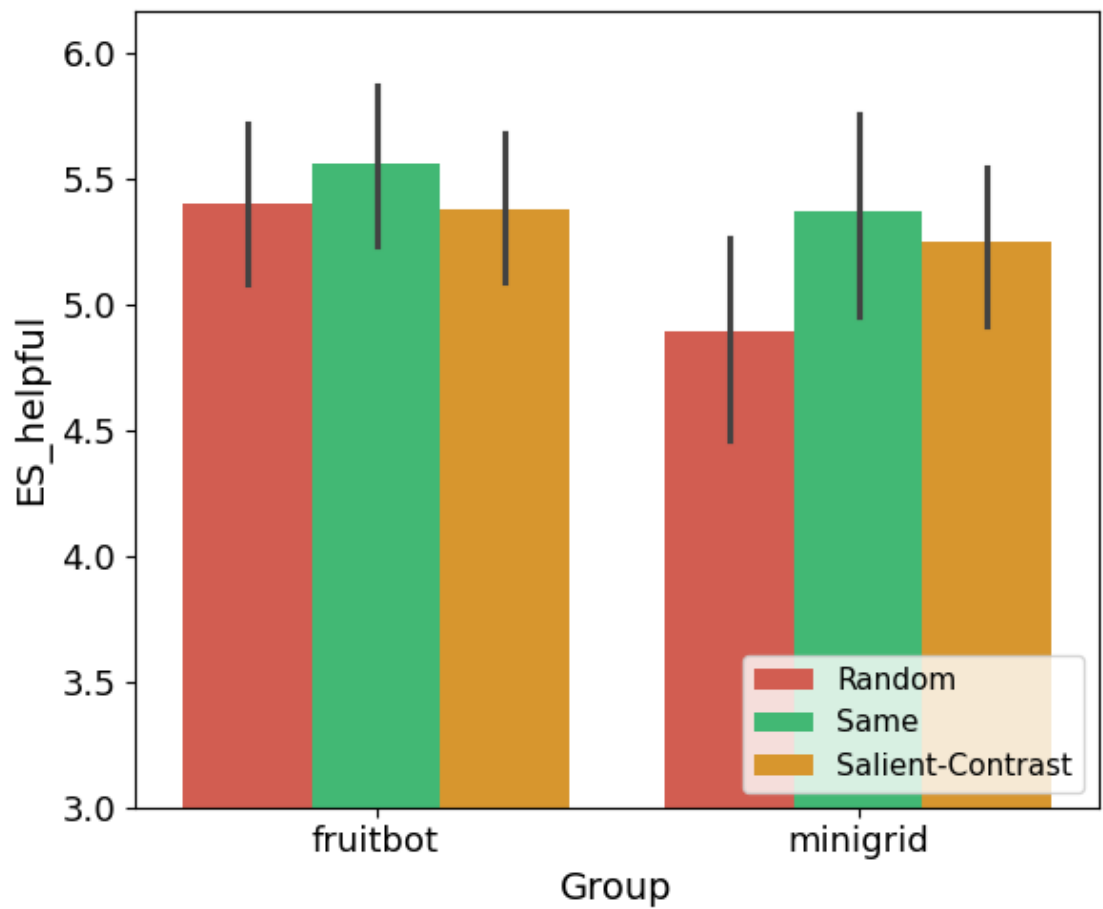
statistic tests for fruitbot on ES_overwhelming:

group_1	group_2	U	p
Random	Salient-Contrast	1423.0	0.5602
Random	Same	1338.0	0.4055
Salient-Contrast	Same	1480.0	0.3606

The visual demonstrations of policy changes helped me understand the differences between agents

Plot values:

group_label	Random	Same	Salient-Contrast
environment			
fruitbot	5.400	5.564	5.375
minigrid	4.895	5.371	5.247



statistic tests for minigrid on ES_helpful:

group_1	group_2	U	p
Random	Salient-Contrast	2558.5	0.2012
Random	Same	1962.0	0.0415
Salient-Contrast	Same	2060.0	0.1791

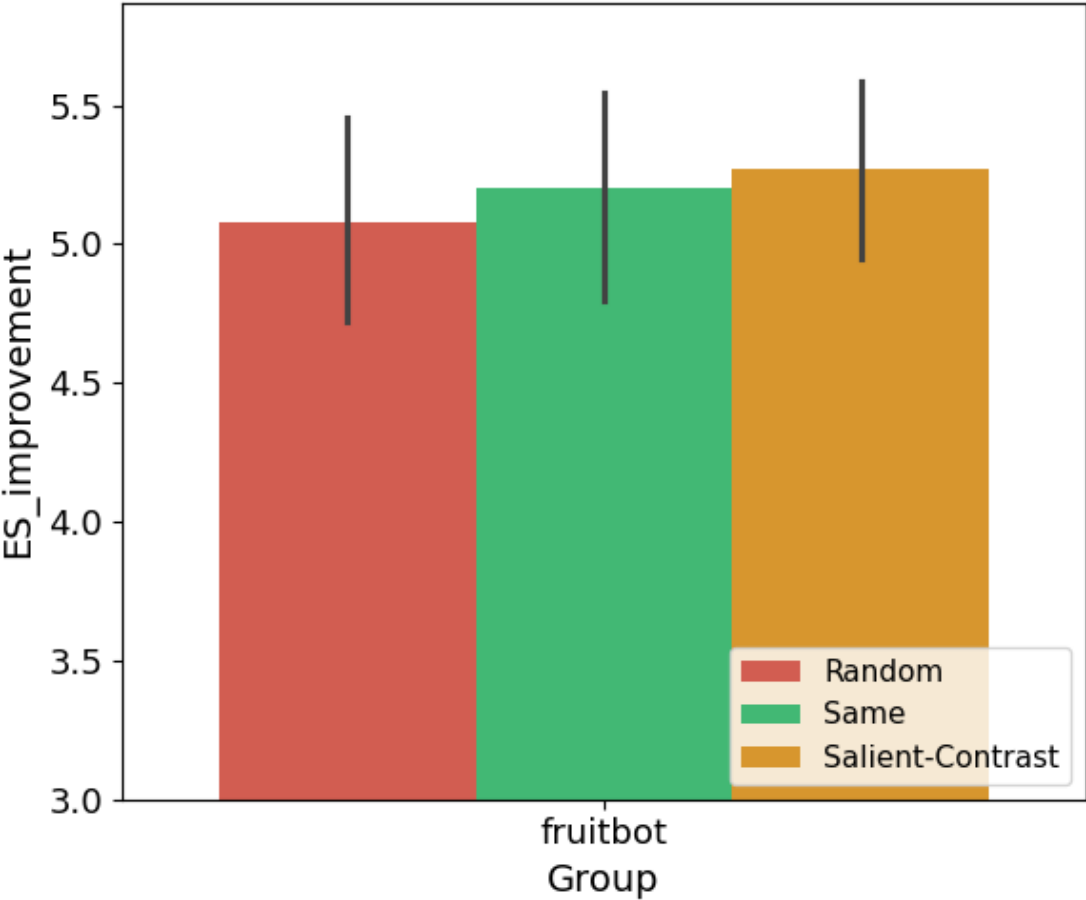
statistic tests for fruitbot on ES_helpful:

group_1	group_2	U	p
Random	Salient-Contrast	1393.5	0.4843
Random	Same	1211.5	0.1361

Salient-Contrast Same 1354.0 0.1208

My feedback helped improve the agent's performance

Plot values:
group_label Random Same Salient-Contrast
environment
fruitbot 5.08 5.2 5.268



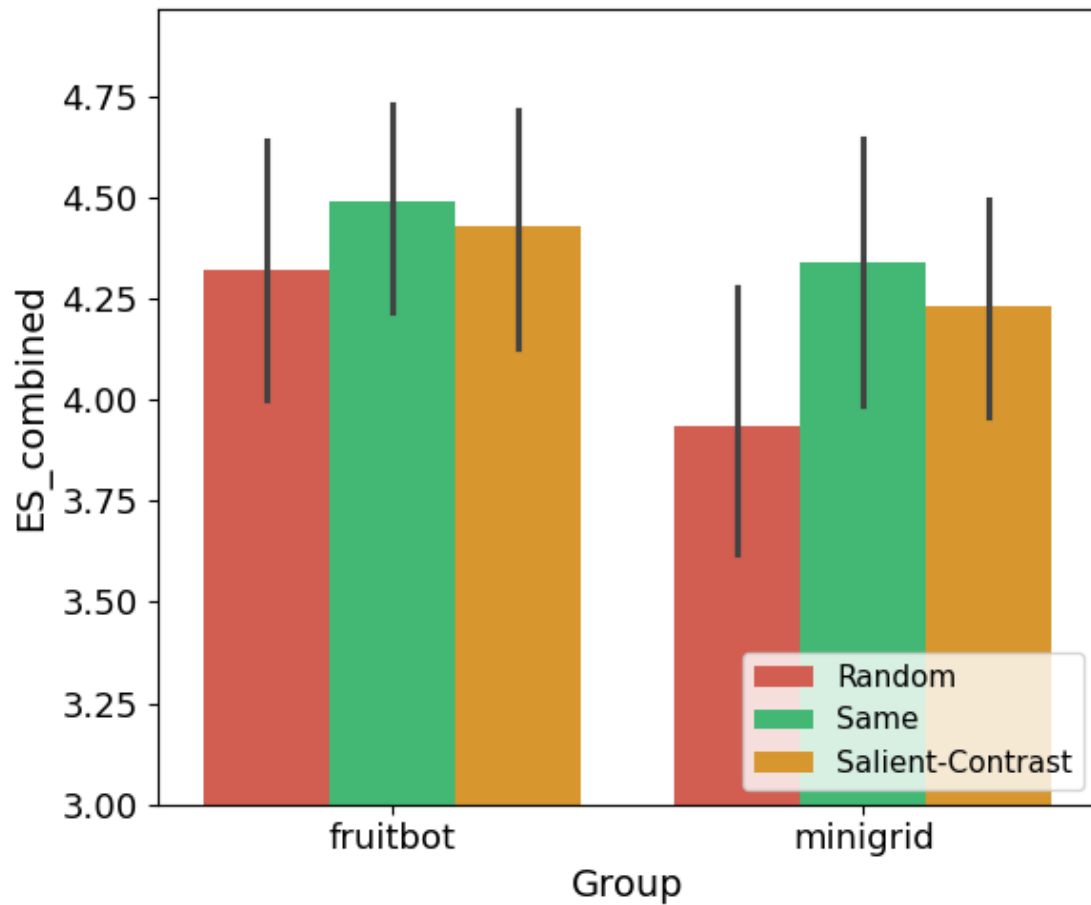
statistic tests for minigrid on ES_improvement:
Empty DataFrame
Columns: []
Index: []
statistic tests for fruitbot on ES_improvement:
group_1 group_2 U p
Random Salient-Contrast 1291.5 0.2413
Random Same 1265.0 0.2335

Salient-Contrast Same 1522.0 0.4572

Combined score of the explanations' satisfaction

Plot values:

group_label	Random	Same	Salient-Contrast
environment			
fruitbot	4.320	4.491	4.429
minigrid	3.934	4.339	4.233



statistic tests for minigrid on ES_combined:

group_1	group_2	U	p
Random	Salient-Contrast	2542.0	0.1832
Random	Same	1993.0	0.0546
Salient-Contrast	Same	2062.0	0.1804

statistic tests for fruitbot on ES_combined:

group_1	group_2	U	p
---------	---------	---	---

Random	Salient-Contrast	1335.5	0.3364
Random	Same	1268.5	0.2369
Salient-Contrast	Same	1491.5	0.3831

```
[91]: # Internal consistency of satisfaction self-report items (Cronbach's alpha) by
      ↪environment
alpha_items = ['ES_feedback_related', 'ES_understandable', 'ES_helpful']

alpha_rows = []
for env in ['minigrid', 'fruitbot']:
    df_env = user_results[user_results['environment'] == env].copy()

    # Build scale with reverse-scored ES_overwhelming.
    scale_df = df_env[alpha_items + ['ES_overwhelming']].copy()
    scale_df['ES_overwhelming_rev'] = 7 - pd.
    ↪to_numeric(scale_df['ES_overwhelming'], errors='coerce')
    scale_df = scale_df[alpha_items + ['ES_overwhelming_rev']].apply(pd.
    ↪to_numeric, errors='coerce')

    # Listwise deletion for alpha computation.
    scale_df = scale_df.dropna()
    n = len(scale_df)
    k = scale_df.shape[1]

    if n < 2 or k < 2:
        alpha = np.nan
    else:
        item_vars = scale_df.var(axis=0, ddof=1)
        total_var = scale_df.sum(axis=1).var(ddof=1)
        alpha = (k / (k - 1)) * (1 - (item_vars.sum() / total_var)) if
        ↪total_var > 0 else np.nan

    alpha_rows.append({'environment': env, 'n_complete': n, 'k_items': k,
    ↪'cronbach_alpha': alpha})

alpha_results = pd.DataFrame(alpha_rows)
print('Cronbach alpha for satisfaction items by environment (ES_overwhelming
    ↪reverse-scored):')
display(alpha_results.round(4))
```

Cronbach alpha for satisfaction items by environment (ES_overwhelming reverse-scored):

	environment	n_complete	k_items	cronbach_alpha
0	minigrid	211	4	0.8252

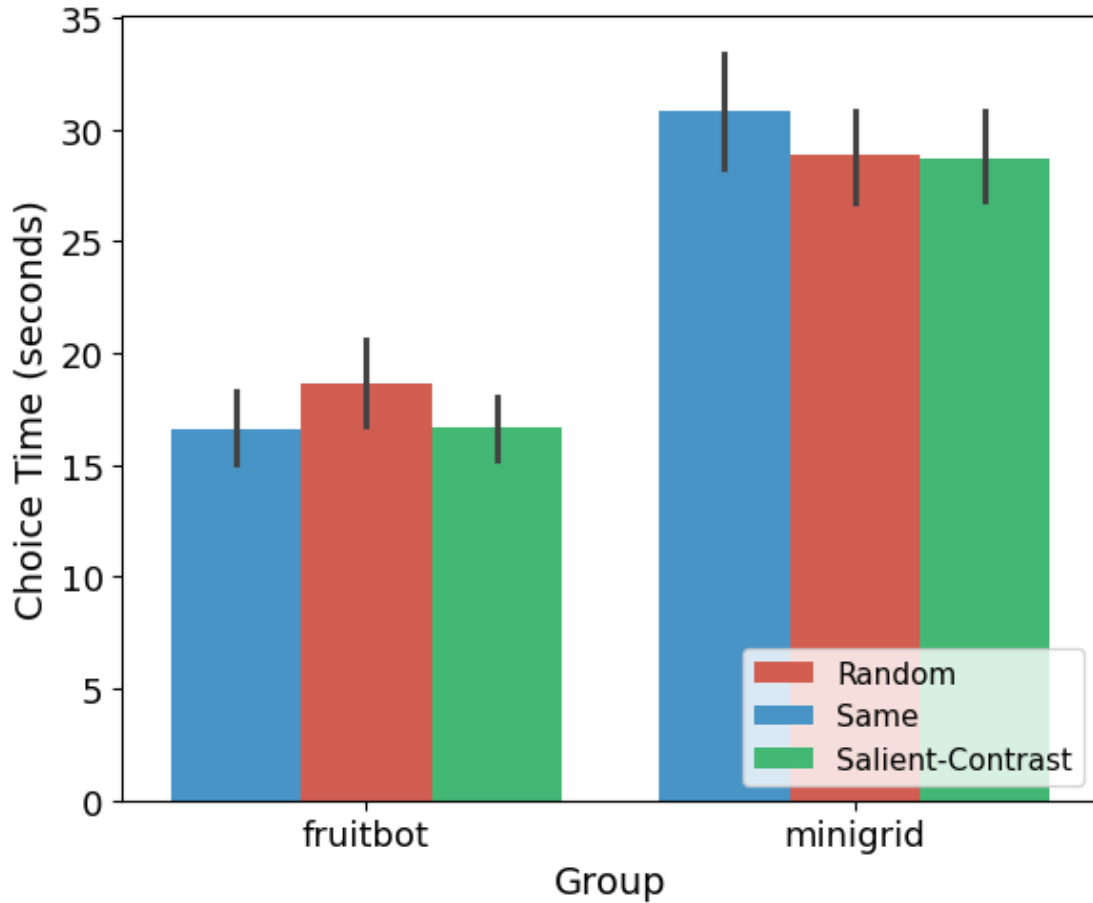
1	fruitbot	161	4	0.7330
---	----------	-----	---	--------

2.4 Choice Time

```
[92]: plot_df = choices_results[(choices_results['choice_time'] < 80) &
    ↪(choices_results['choice_time'] > 0)] # Filter out extreme outliers for
    ↪better visualization
if plot_df.empty:
    print("No valid choice_time data to plot.")
else:
    plot_barplot(plot_df, 'environment', 'choice_time', y_label='Choice Time
    ↪(seconds)',
                  hue='group_label', palette=colors_palette,
                  bar_label_fmt='%.1f', estimator='mean')
    for env in ['minigrid', 'fruitbot']:
        print(f"statistic tests for {env}:")
        anova_test(plot_df[plot_df['environment'] == env], 'choice_time',
    ↪'group_label')
        mannwhitneyu_test(plot_df[plot_df['environment'] == env],
    ↪'choice_time', 'group_label')
```

Plot values:

group_label	Random	Salient-Contrast	Same
environment			
fruitbot	18.611	16.687	16.598
minigrid	28.888	28.706	30.865



statistic tests for minigrid:

ANOVA $F=1.0234$, $p=0.3600$ | Kruskal-Wallis $H=1.6028$, $p=0.4487$

group_1	group_2	U	p
Random	Salient-Contrast	25696.5	0.5835
Random	Same	21588.0	0.2102
Salient-Contrast	Same	18685.5	0.4642

statistic tests for fruitbot:

ANOVA $F=1.7022$, $p=0.1832$ | Kruskal-Wallis $H=1.8955$, $p=0.3876$

group_1	group_2	U	p
Random	Salient-Contrast	17820.0	0.4608
Random	Same	18380.0	0.1695
Salient-Contrast	Same	19646.5	0.5094

2.5 Delegation - Trust the Agent

```
[93]: # Plot: Delegate to Agent by Group
plot_barplot(user_results[user_results['final_agent'] > 0], 'environment',
             hue='trust_the_agent', errorbar='ci', x_label='Environment',
```

```

        y_label='Delegate to Agent (%)', palette=colors_palette,
        bar_label_fmt='%.0f%%', save_path='delegate_to_agent_by_group.pdf')

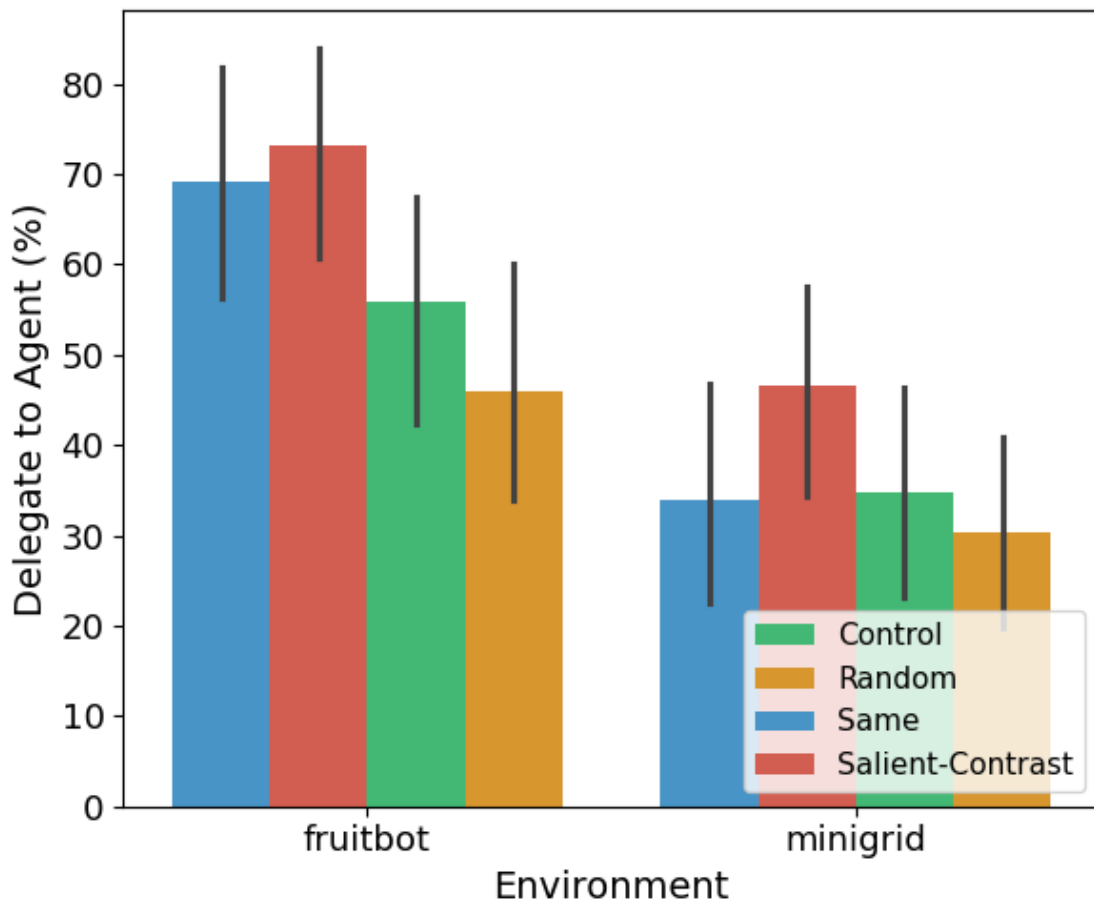
for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env} on trust_the_agent:")
    anova_test(user_results[user_results['environment'] == env],
        'trust_the_agent', 'group_label')

    mannwhitneyu_test(user_results[user_results['environment'] == env],
        'trust_the_agent', 'group_label', test_type='two-sided')

```

Plot values:

group_label	Control	Random	Salient-Contrast	Same
environment				
fruitbot	55.769	46.000	73.214	69.091
minigrid	34.783	30.263	46.575	33.871



statistic tests for minigrid on trust_the_agent:

ANOVA F=1.5872, p=0.1927 | Kruskal-Wallis H=4.7318, p=0.1925

group_1	group_2	U	p
Control	Random	2740.5	0.5645
Control	Salient-Contrast	2221.5	0.1552
Control	Same	2158.5	0.9152
Random	Salient-Contrast	2321.5	0.0415
Random	Same	2271.0	0.6541
Salient-Contrast	Same	2550.5	0.1365

statistic tests for fruitbot on trust_the_agent:

ANOVA F=3.5685, p=0.0150 | Kruskal-Wallis H=10.3302, p=0.0160

group_1	group_2	U	p
Control	Random	1427.0	0.3281
Control	Salient-Contrast	1202.0	0.0595
Control	Same	1239.5	0.1576
Random	Salient-Contrast	1019.0	0.0045
Random	Same	1057.5	0.0173
Salient-Contrast	Same	1603.5	0.6358

2.6 Final Agent

```
[94]: # Plot: Count of Final Agent by Group (side-by-side: Minigrid and Fruitbot)
ordered_groups = ['Control', 'Random', 'Same', 'Salient-Contrast']
fig, axes = plt.subplots(1, 2, figsize=(13, 5), sharey=True)

for ax, env in zip(axes, ['minigrid', 'fruitbot']):
    grouped = (
        user_results[user_results['environment'] == env]
        .groupby(['group_label', 'final_agent'])
        .size()
        .reset_index(name='count')
    )

    sns.barplot(
        data=grouped,
        x='group_label',
        y='count',
        hue='final_agent',
        order=ordered_groups,
        palette='Set2',
        ax=ax
    )

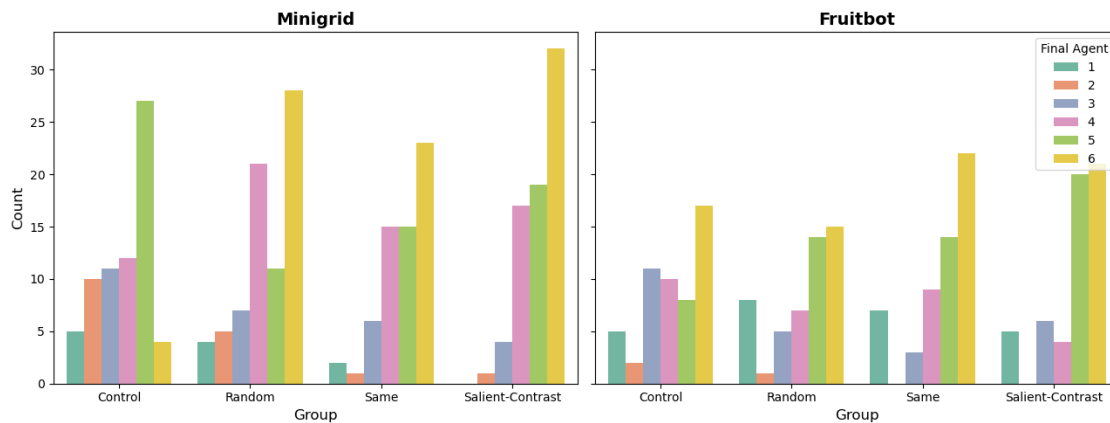
    ax.set_title('Minigrid' if env == 'minigrid' else 'Fruitbot', fontsize=14,
        fontweight='bold')
    ax.set_xlabel('Group', fontsize=12)
    ax.set_ylabel('Count' if env == 'minigrid' else '', fontsize=12)
    ax.tick_params(axis='x', rotation=0)
```

```

# Keep legend only on Fruitbot (right panel), upper-right
axes[0].legend_.remove()
handles, labels = axes[1].get_legend_handles_labels()
axes[1].legend(handles=handles, labels=labels, title='Final Agent', loc='upper_
    ↪right')

plt.tight_layout()
plt.savefig(f"{out_dir}/count_of_final_agent_by_group.pdf", bbox_inches='tight')
plt.show()

```



```

[95]: # Final agent score by group and trust the agent

plot_barplot(user_results, 'environment', 'final_agent_mean_score',
    ↪hue='group_label', hue_order=group_names,
            x_label='Environment',
            y_label='Final Agent Mean Score',
            estimator = 'mean', bar_label_fmt='%.2f',
            palette=colors_palette,
            save_path='final_agent_mean_score_by_group.pdf')

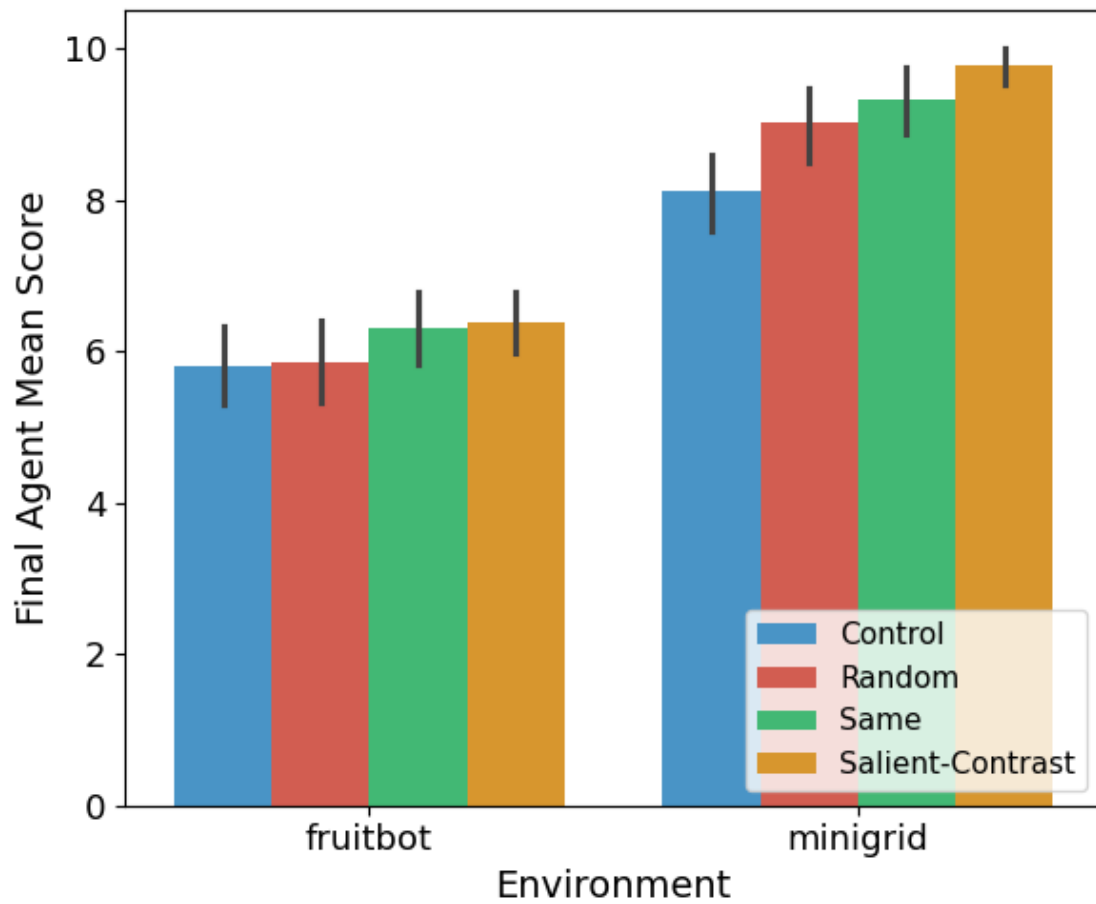
# statistic tests for final agent score by group
for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env}:")
    anova_test(user_results[user_results['environment'] == env],
    ↪'final_agent_mean_score', 'group_label')
    mannwhitneyu_test(user_results[user_results['environment'] == env],
    ↪'final_agent_mean_score', 'group_label', test_type='less')

```

Plot values:

group_label Control Random Same Salient-Contrast

environment				
fruitbot	5.805	5.858	6.309	6.388
minigrid	8.111	9.022	9.333	9.767



statistic tests for minigrid:

ANOVA $F=10.0882$, $p=0.0000$ | Kruskal-Wallis $H=27.9999$, $p=0.0000$

group_1	group_2	U	p
Control	Random	1926.0	0.0024
Control	Salient-Contrast	1278.5	0.0000
Control	Same	1346.5	0.0001
Random	Salient-Contrast	2249.5	0.0185
Random	Same	2165.5	0.1988
Salient-Contrast	Same	2521.5	0.8856

statistic tests for fruitbot:

ANOVA $F=1.4569$, $p=0.2274$ | Kruskal-Wallis $H=4.5913$, $p=0.2043$

group_1	group_2	U	p
Control	Random	1274.0	0.3662
Control	Salient-Contrast	1204.0	0.0398
Control	Same	1212.0	0.0598

Random	Salient-Contrast	1183.5	0.0775
Random	Same	1196.0	0.1176
Salient-Contrast	Same	1570.5	0.5760

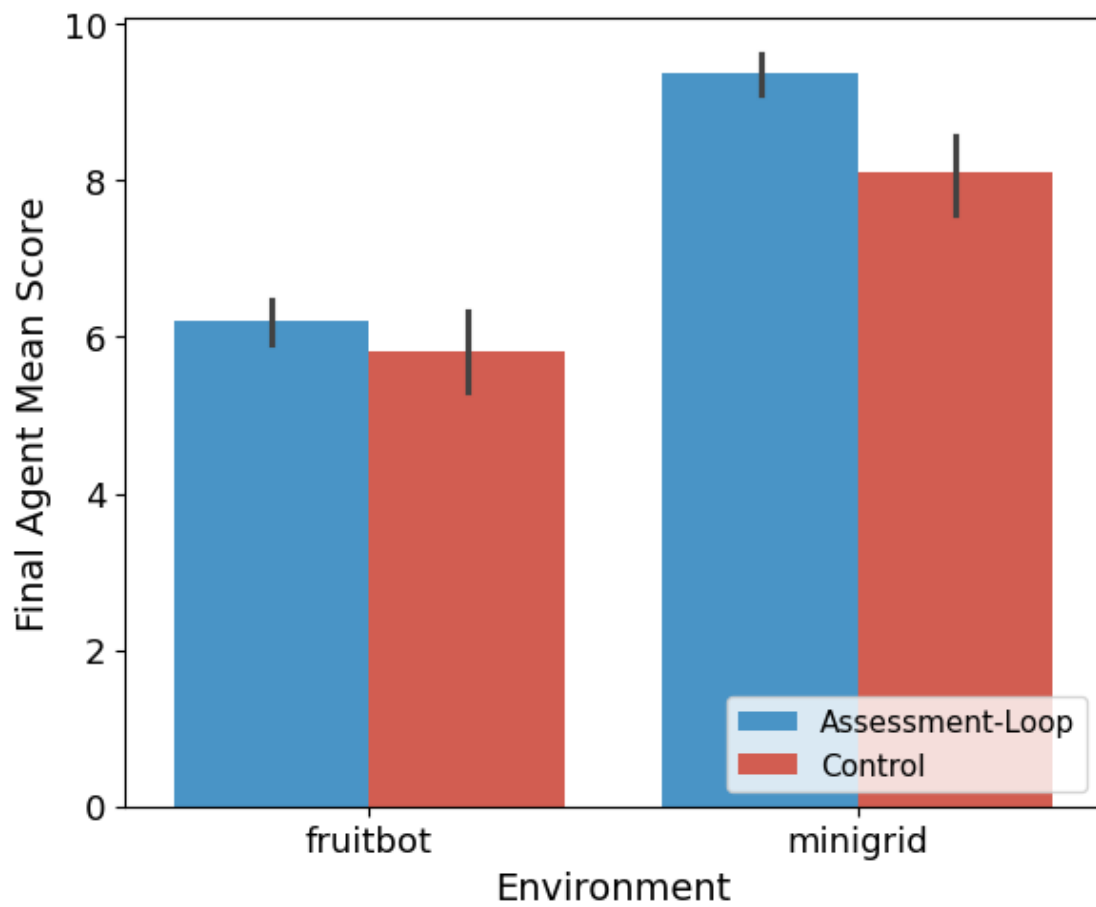
```
[96]: plot_df = user_results.copy()
plot_df['group_binary'] = np.where(plot_df['group_label'] == 'Control',
    ↪ 'Control', 'Assessment-Loop')

plot_barplot(
    plot_df,
    'environment',
    'final_agent_mean_score',
    hue='group_binary',
    estimator='mean',
    x_label='Environment',
    y_label='Final Agent Mean Score',
    palette=['#3498db', '#e74c3c'],
    bar_label_fmt='%.2f',
    save_path='final_agent_mean_score_control_vs_assessment_loop.pdf'
)

for env in ['minigrid', 'fruitbot']:
    print(f"statistic tests for {env}:")
    mannwhitneyu_test(plot_df[plot_df['environment'] == env],
    ↪ 'final_agent_mean_score', 'group_binary', test_type='two-sided')
```

Plot values:

group_binary	Assessment-Loop	Control
environment		
fruitbot	6.196	5.805
minigrid	9.371	8.111



```

statistic tests for minigrid:
  group_1 group_2      U    p
Assessment-Loop Control 10008.0 0.0
statistic tests for fruitbot:
  group_1 group_2      U    p
Assessment-Loop Control 4843.0 0.1273

```